

Introduction to PyTorch



What is PyTorch?



- ❑ PyTorch is an **open-source** deep learning framework developed by Facebook's AI Research lab (FAIR) <https://pytorch.org/>
- ❑ PyTorch provides a **flexible** and efficient platform for building and training neural networks.
- ❑ It was first released in 2016 and has since become **one of the most widely used frameworks in academia and industry.**

What PyTorch helps with?

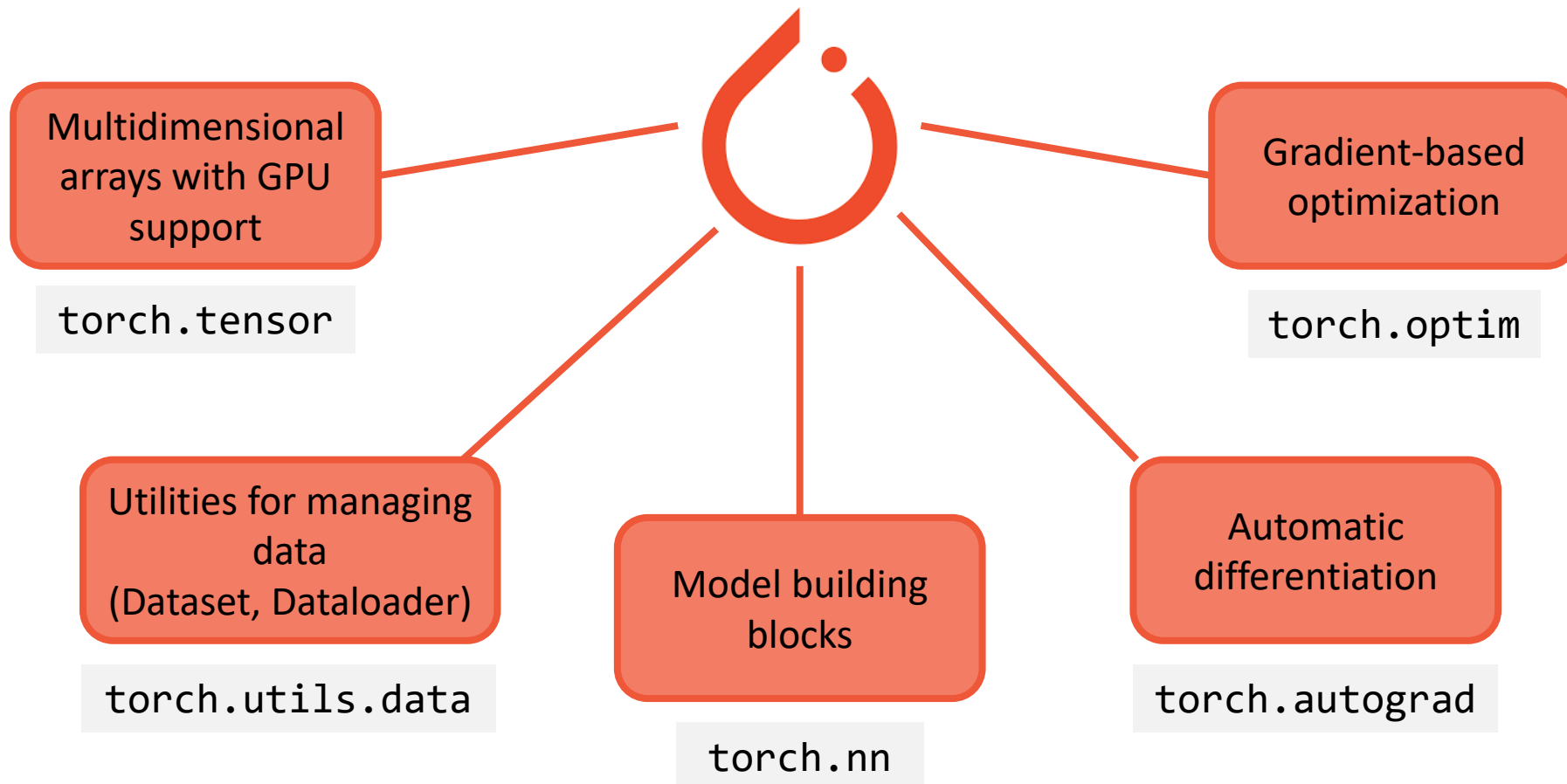


- ❑ Explicit code, flexibility
- ❑ Large community
- ❑ Production-ready deployment
- ❑ Domain-specific extensions : TorchVision, TorchText, TorchAudio
- ❑ Integration with Hugging Face Transformers for NLP and generative AI



Hugging Face

What PyTorch helps with?



Other libraries

NumPy: Scientific computing package

<https://numpy.org/>



Scikit-Learn: Classical machine learning algorithms and preprocessing. Built on NumPy, SciPy, and Matplotlib

<https://scikit-learn.org/stable/>



Matplotlib: Visualization library for static plots. <https://matplotlib.org/>



Seaborn: high level library, built on matplotlib.

<https://seaborn.pydata.org/>



What is Google Colab?



- ❑ Google Colab is a web IDE (Integrated Development Environment) for Python
<https://colab.research.google.com/>
- ❑ Jupyter Notebook environment directly in the browser, connected to cloud servers
- ❑ Environment is already setup, with popular ML/DL packages (PyTorch, TensorFlow, scikit-learn, pandas, etc.)
- ❑ Unlimited access to CPU, limited but free access to GPU (Nvidia T4)
- ❑ Enables access to Google Drive



Time to practice!



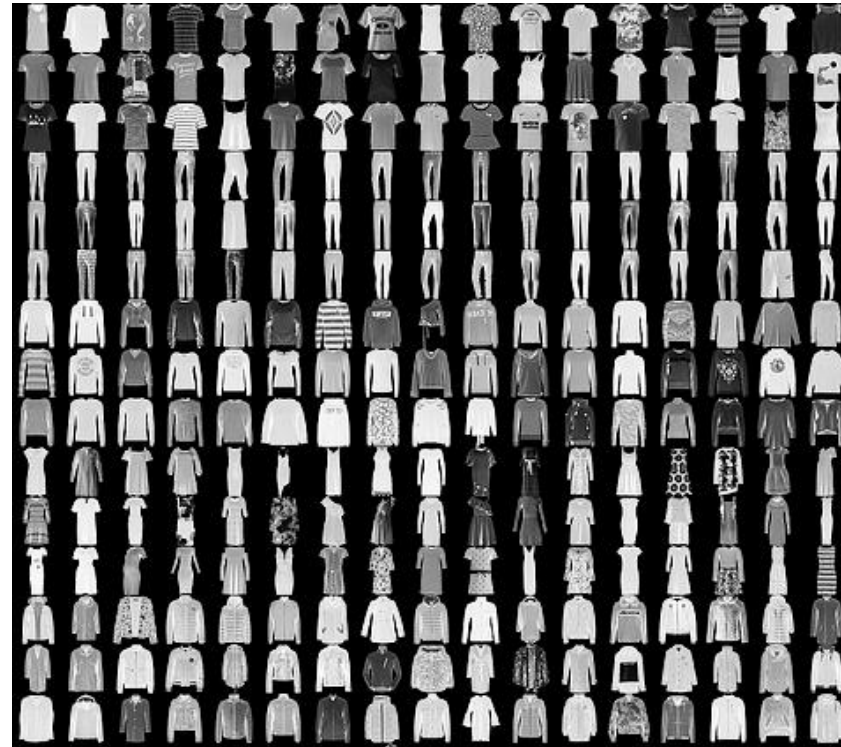
Common Data Science Datasets

- ❑ MNIST: Handwritten digits
- ❑ [PyTorch Datasets](#)



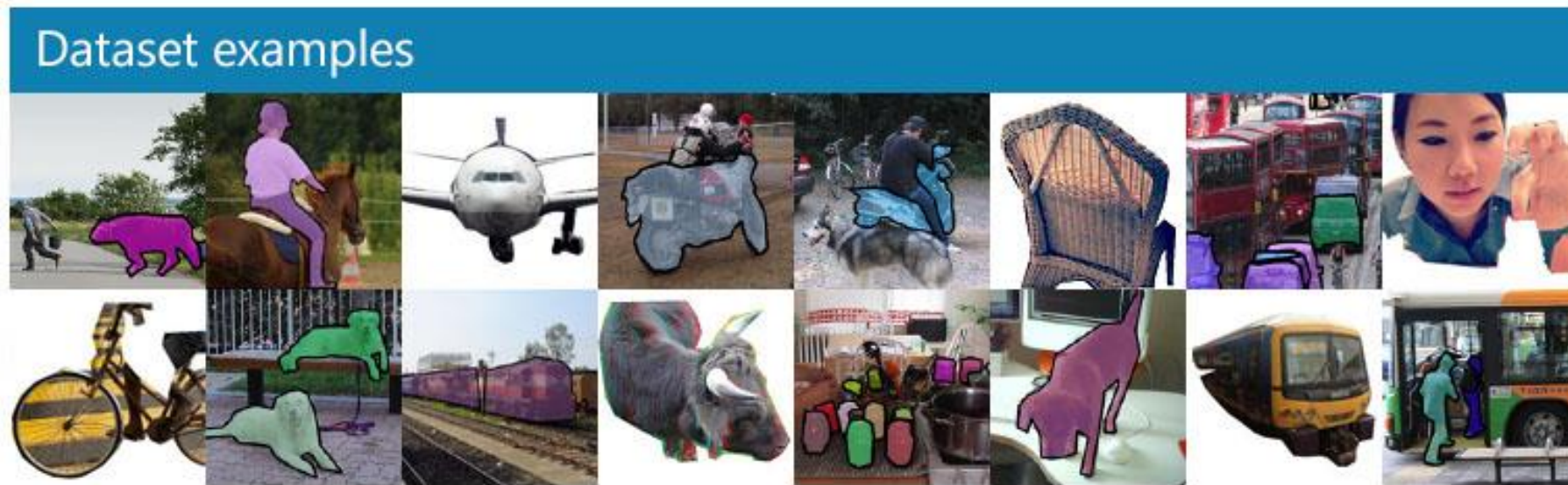
Common Data Science Datasets

- ❑ [Fashion-MNIST](#): Zalando fashion item classification
- ❑ [PyTorch Datasets](#)



Common Data Science Datasets

- ❑ [COCO](#): : Common Objects in COntext
- ❑ [PyTorch Datasets](#)



Common Data Science Datasets

- ❑ [CIFAR-10 / CIFAR-100](#): Object classification in small images
- ❑ [PyTorch Datasets](#)

airplane



automobile



bird



cat



deer



dog



frog



horse



ship



truck



Common Data Science Datasets

- ❑ [ImageNet](#): Large-scale visual recognition
- ❑ [PyTorch Datasets](#)



Common Data Science Datasets

- ❑ Wine dataset: Wine quality classification
- ❑ [Scikit-Learn Datasets](#)



	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5

Common Data Science Datasets

- ❑ [Amazon Review Dataset](#): Huge dataset for sentiment analysis and recommendation systems
- ❑ [Hugging Face Datasets](#)

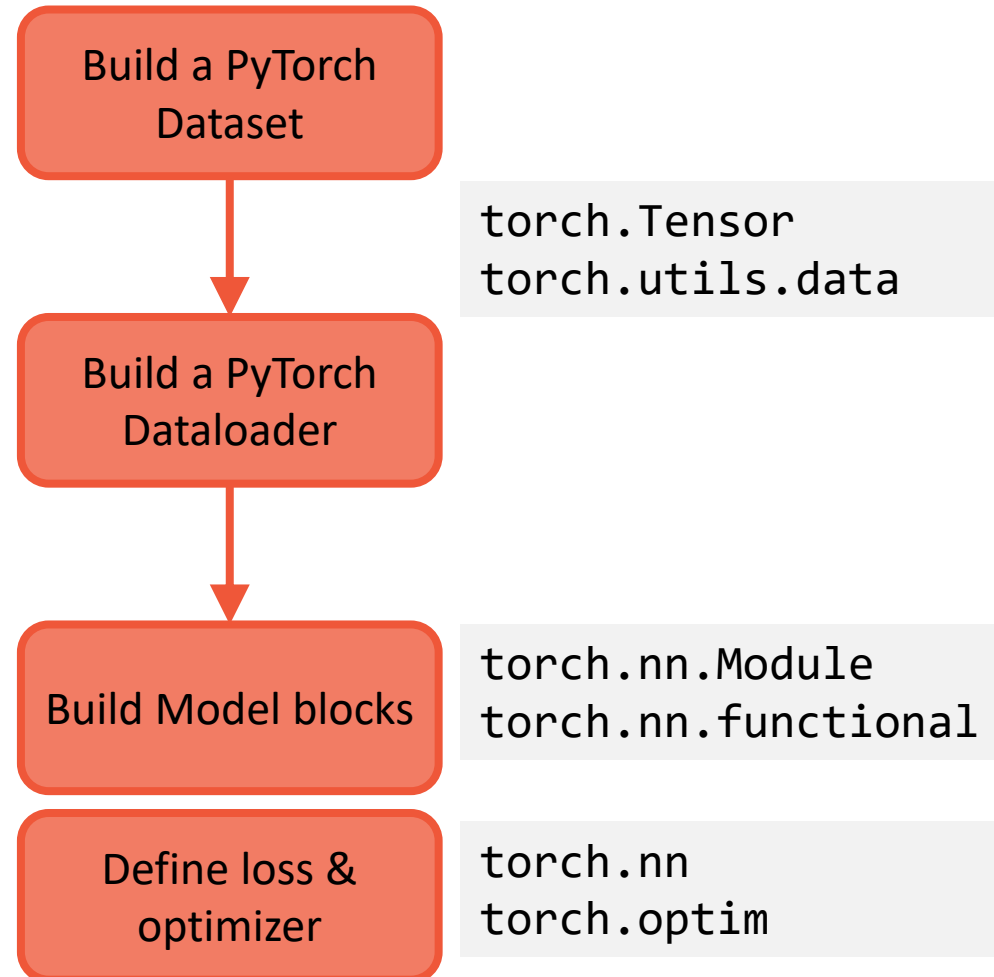
	Product Name	Brand Name	Price	Rating	Reviews	Review Votes
82379	ASUS ZenFone 3 MAX 5.2-inch (2GB RAM, 16GB sto...	NaN	149.99	5	Faster, good quality camera and the fingerprin...	6.0
232360	LG Nexus 5X Unlocked Smartphone with 5.2-Inch ...	LG	399.00	5	Love the cell very good one in every thing	0.0
39160	Apple iPhone 5c Factory Unlocked Cellphone, 8G...	NaN	199.99	5	The Phone came unlocked & its a sweet surprise...	0.0
166311	CHSLING Unlocked Android MTK6580 Quad Core ROM...	NaN	79.99	3	Its fairly ok, but really not a high end phone...	0.0
231945	LG Nexus 5X Unlocked Smartphone - White 32GB (...)	LG	399.00	1	Thus phone worked great for about 3 months. Th...	7.0
30001	Apple iPhone 5c 32GB (Blue) - AT&T	Apple	274.95	5	What an upgrade compared to the iPhone 4. Goin...	7.0
313198	Samsung Galaxy Grand Prime DUOS G531H/DS - Gra...	Samsung	179.99	4	I liked it at first but is starting to lag alr...	0.0
138219	BLU Studio 5.0 C HD Unlocked Cellphone, White	BLU	2000.00	4	very nice	0.0
66571	Apple iPhone 6s 64 GB International Warranty U...	Apple	689.95	1	It is not a new one. The tagboard on the box w...	0.0
109303	BLU Dash J Unlocked Phone - Retail Packaging -...	BLU	39.99	1	This phone was truly a terrible purchase!! It ...	1.0

Where to find datasets for free?

- ❑ [PyTorch Datasets](#)
- ❑ [Keras Datasets](#) (only 7)
- ❑ [Scikit-Learn Datasets](#)
- ❑ [Kaggle Datasets](#)
- ❑ [Hugging Face Datasets](#)
- ❑ [Google Dataset Search](#)

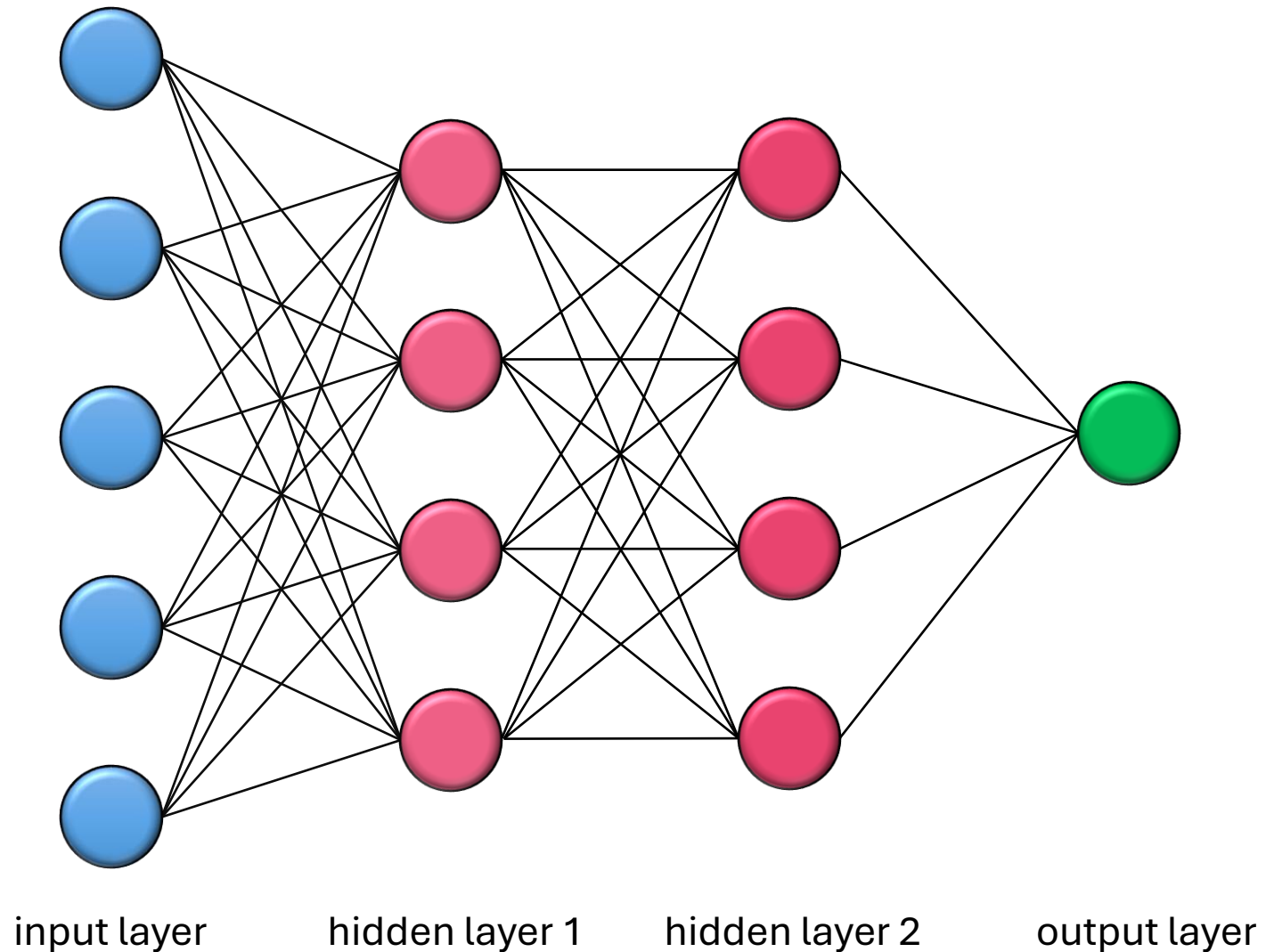


PyTorch data and model setup



- ❑ The Dataset object transforms data in a format that PyTorch can handle
- ❑ Dataloader object determines how to feed the model with data (mini batch, shuffling or not, etc.)
- ❑ Model object defines structure: layers, activation functions, etc.
- ❑ Criterion & optimizer define how loss and parameter optimization is computed

Neural network representation: layers & connexions



Neural networks are essentially matrix multiplications + activations

$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Input vector

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \\ w_{41} & w_{42} & w_{43} \end{bmatrix}$$

Weight matrix

$$b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}$$

Bias vector

One
hidden
layer

Linear relationship

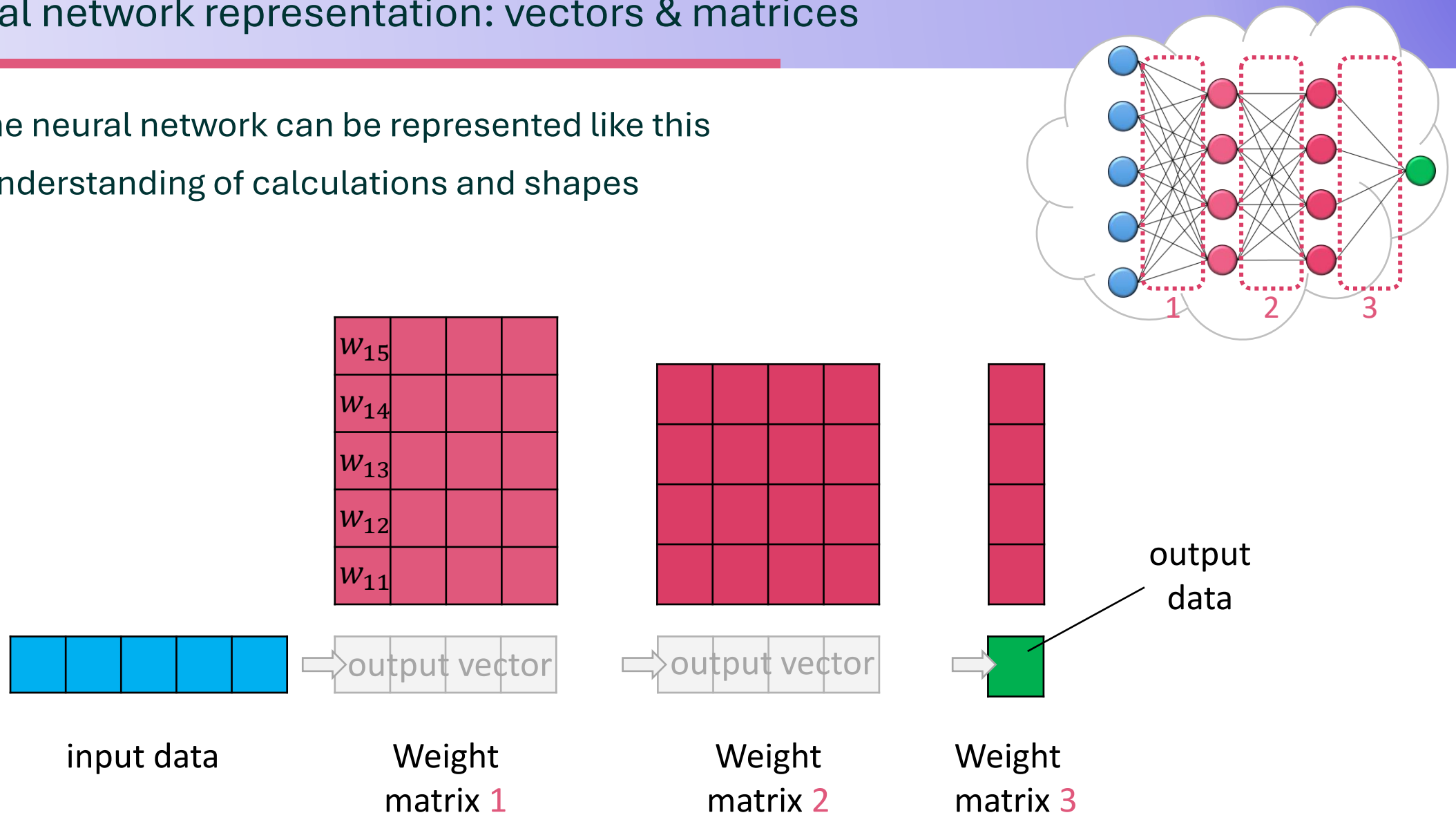
$$Z = WX + b = \begin{bmatrix} w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1 \\ w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2 \\ w_{31}x_1 + w_{32}x_2 + w_{33}x_3 + b_3 \\ w_{41}x_1 + w_{42}x_2 + w_{43}x_3 + b_4 \end{bmatrix}$$

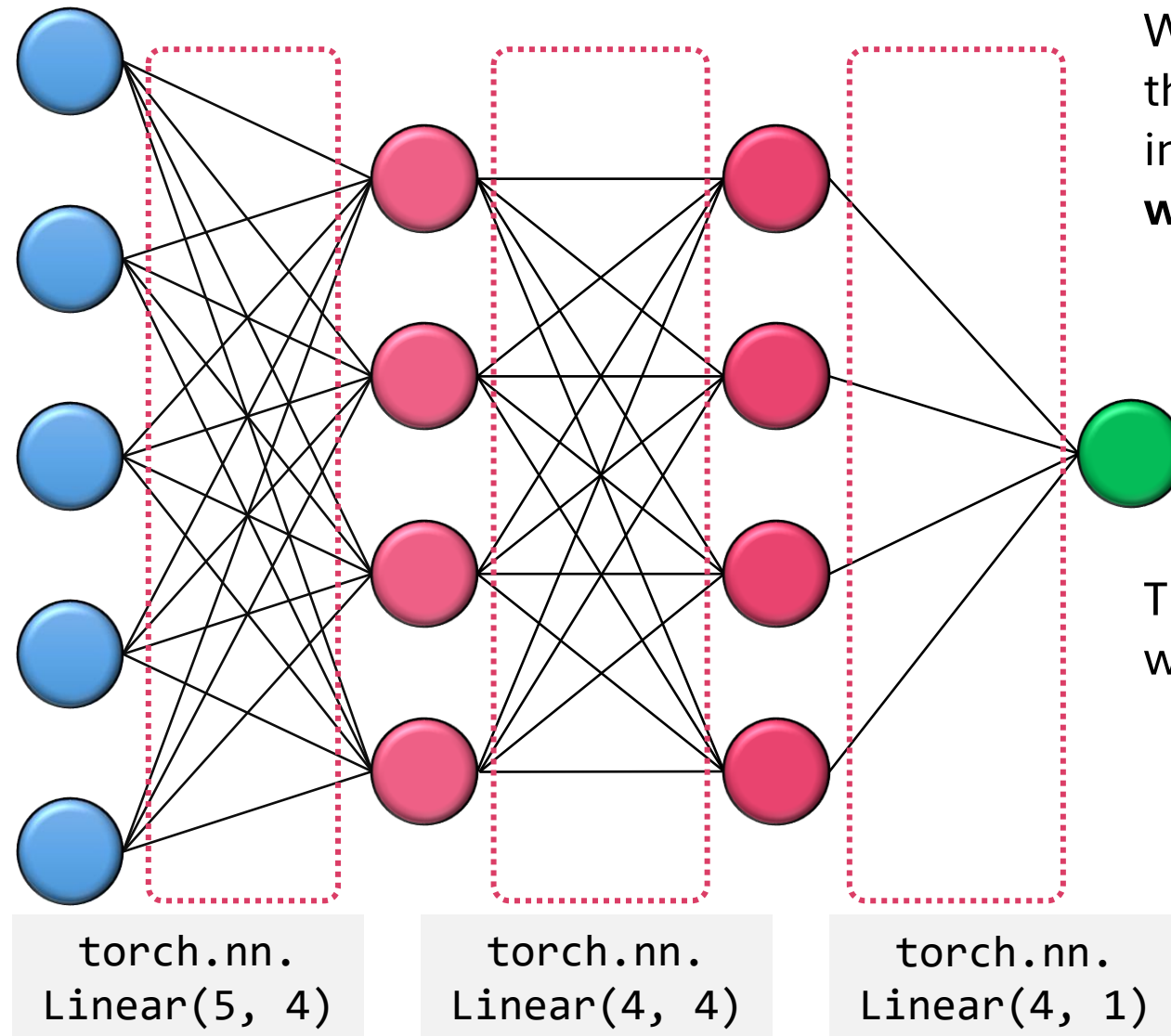
Activation
(layer output)

$$A = f(Z) = [z_1 \quad z_2 \quad z_3]$$

Neural network representation: vectors & matrices

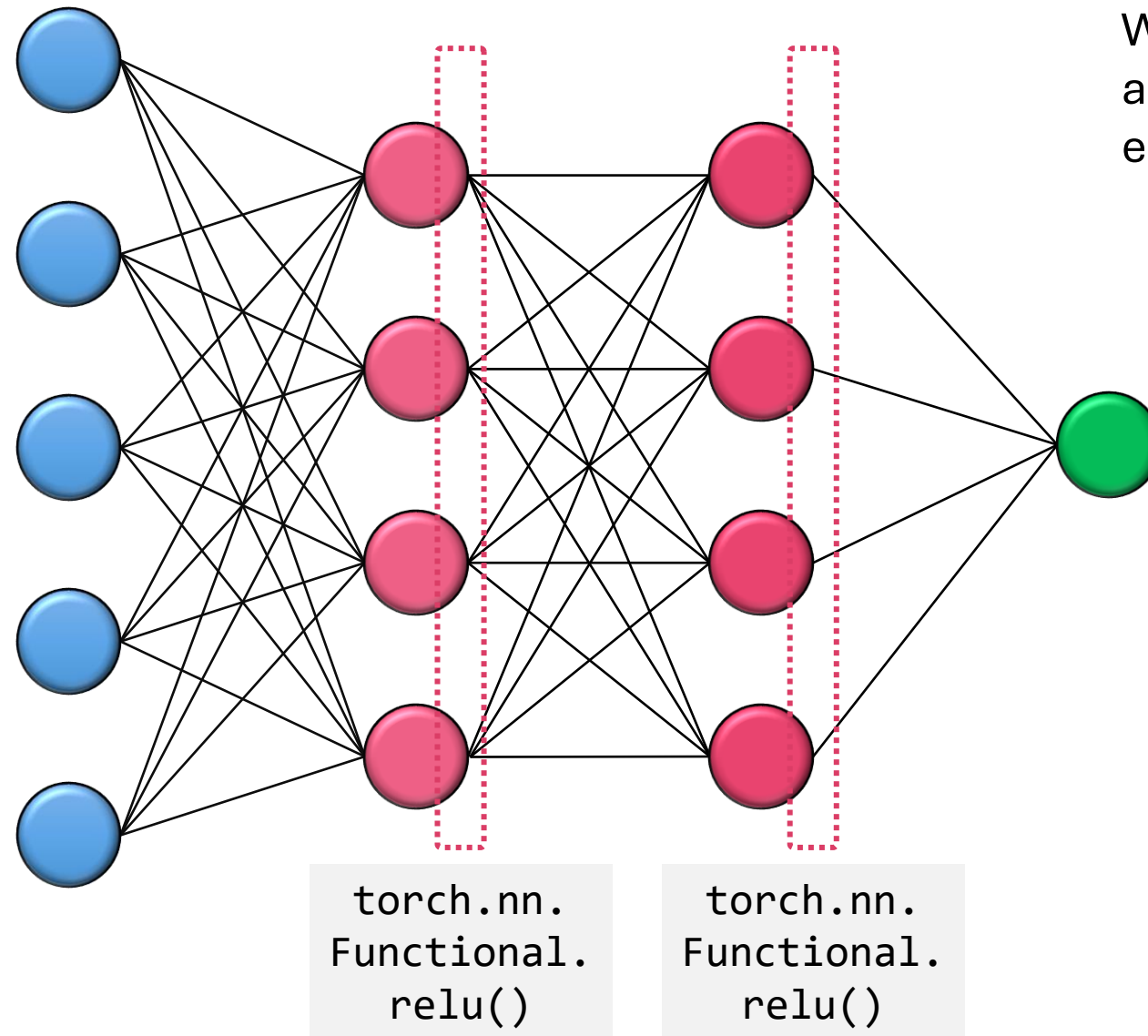
- ❑ The same neural network can be represented like this
- ❑ Better understanding of calculations and shapes



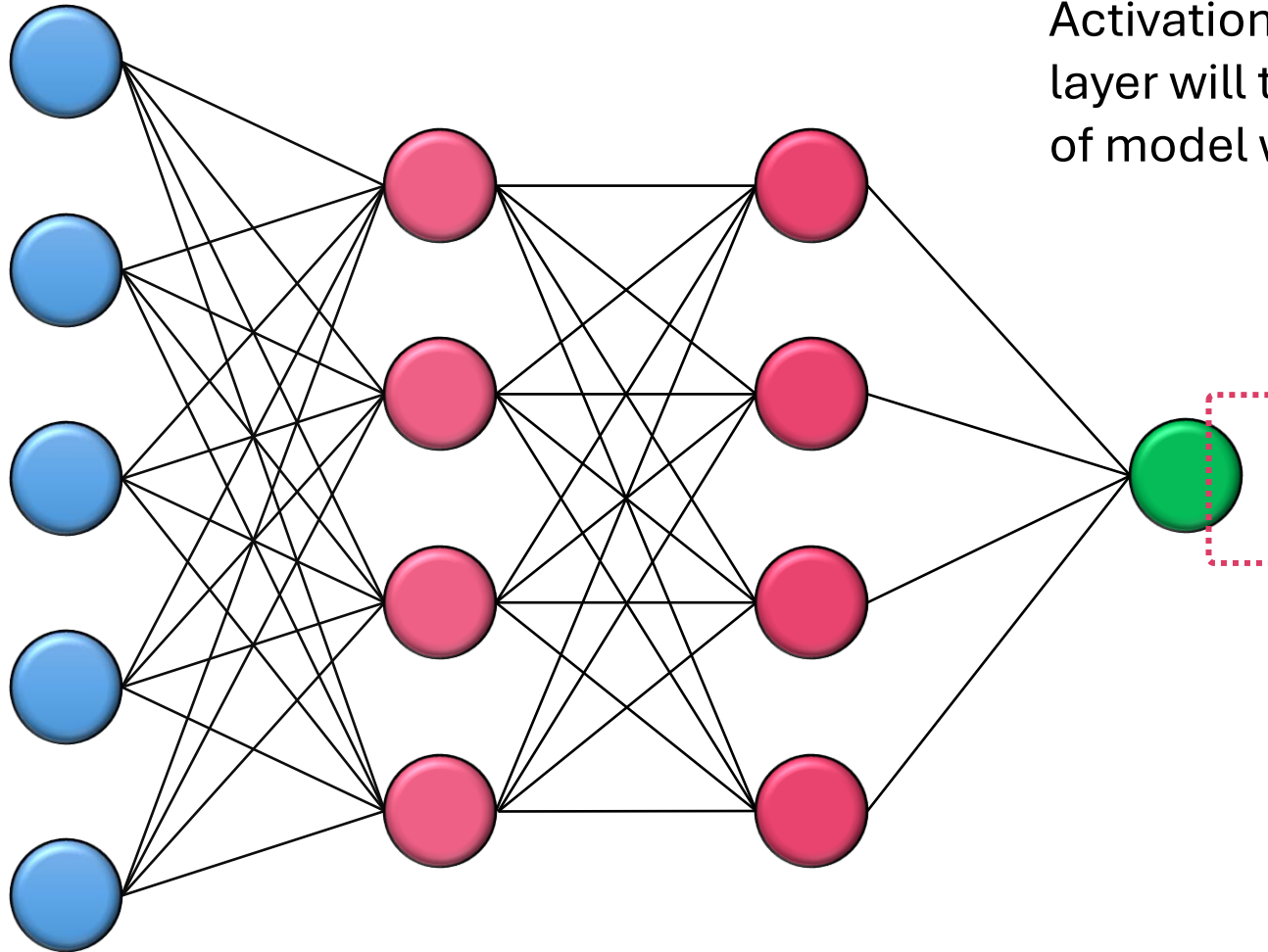


We are going to put each of these fully connected layers into layer objects which are **weight matrices**

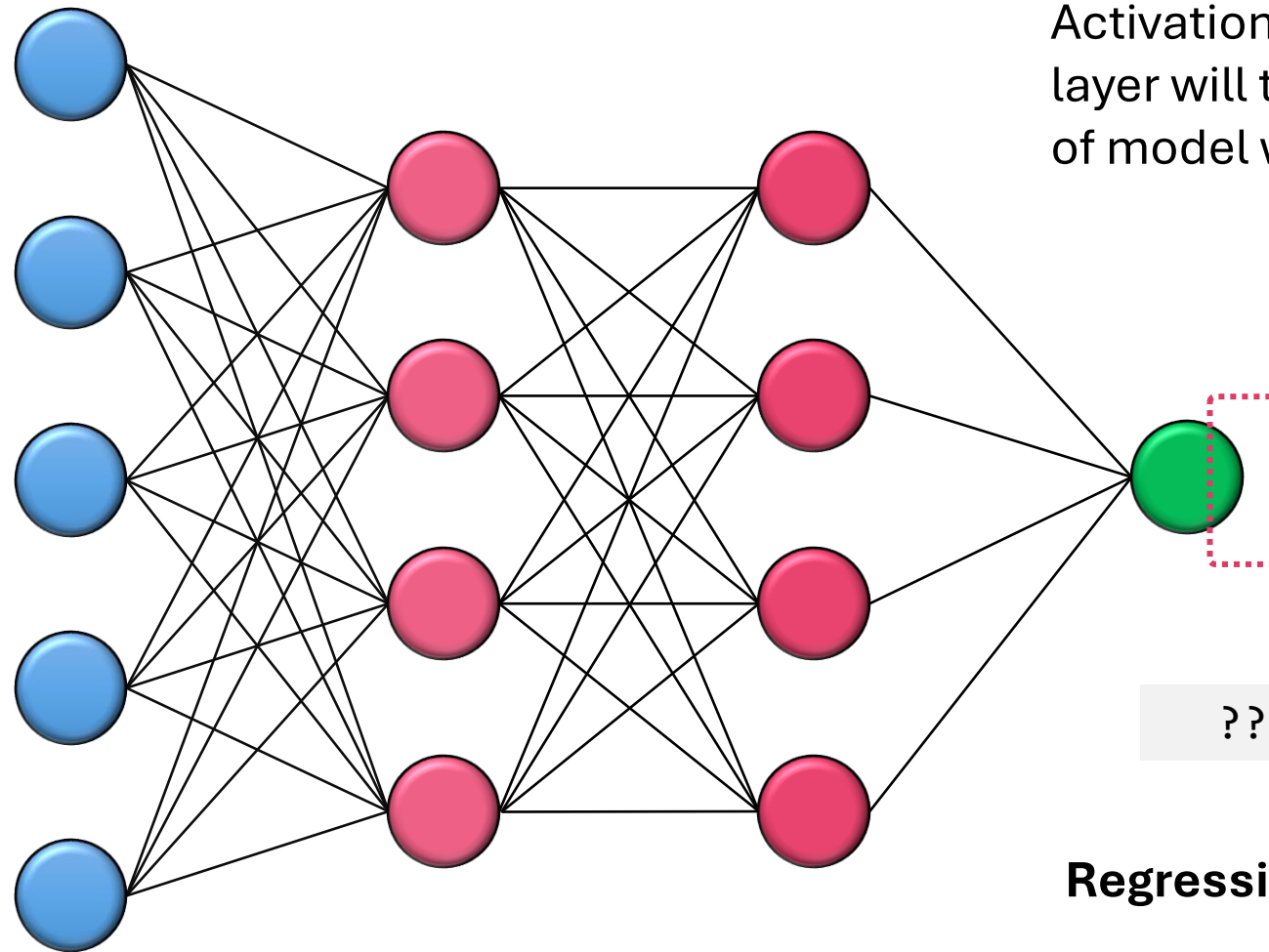
The model will learn these weight matrices



We are also going to define activation functions for each node output



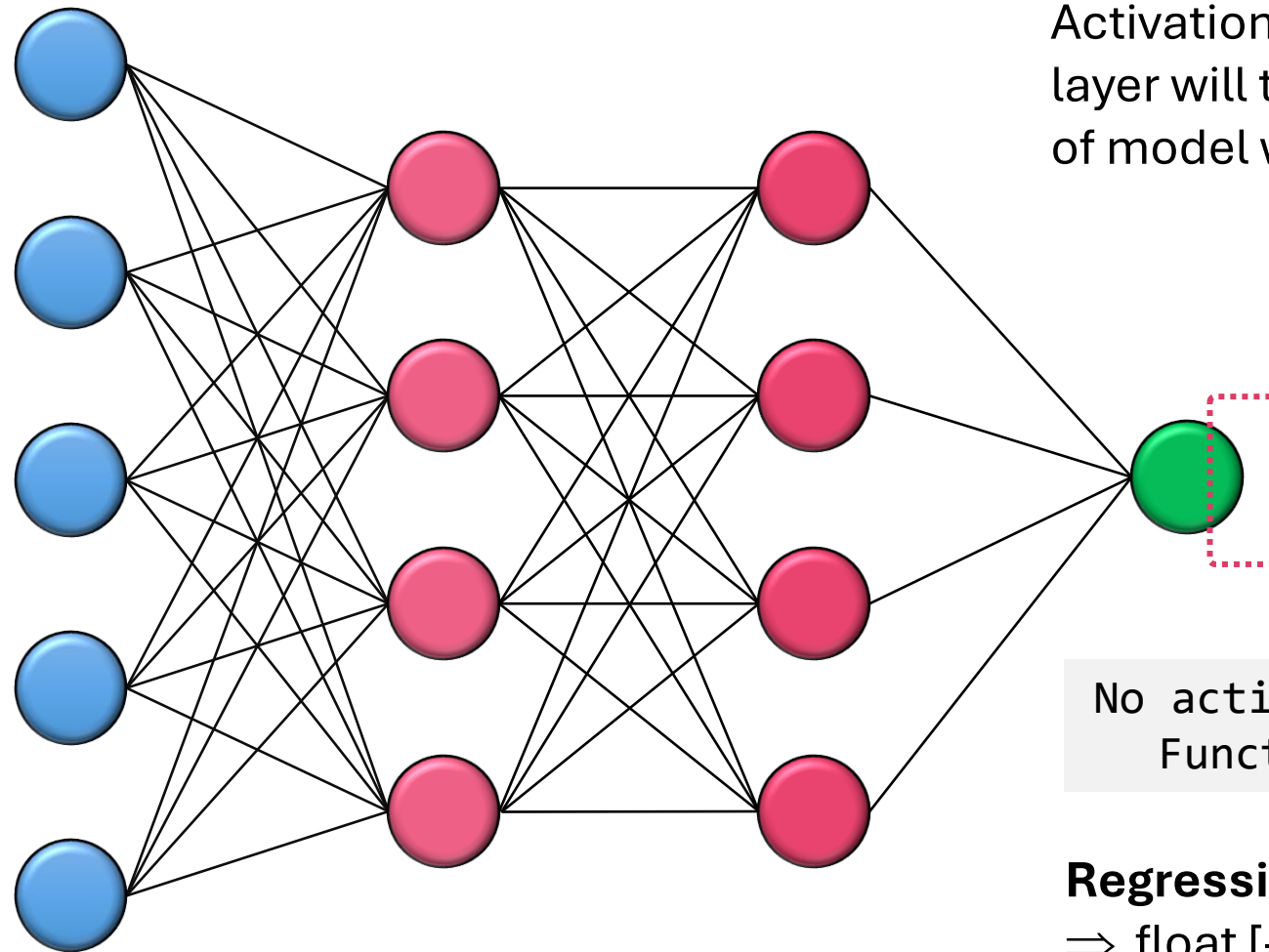
Activation function for the output layer will then define which type of model we want



Activation function for the output layer will then define which type of model we want

??

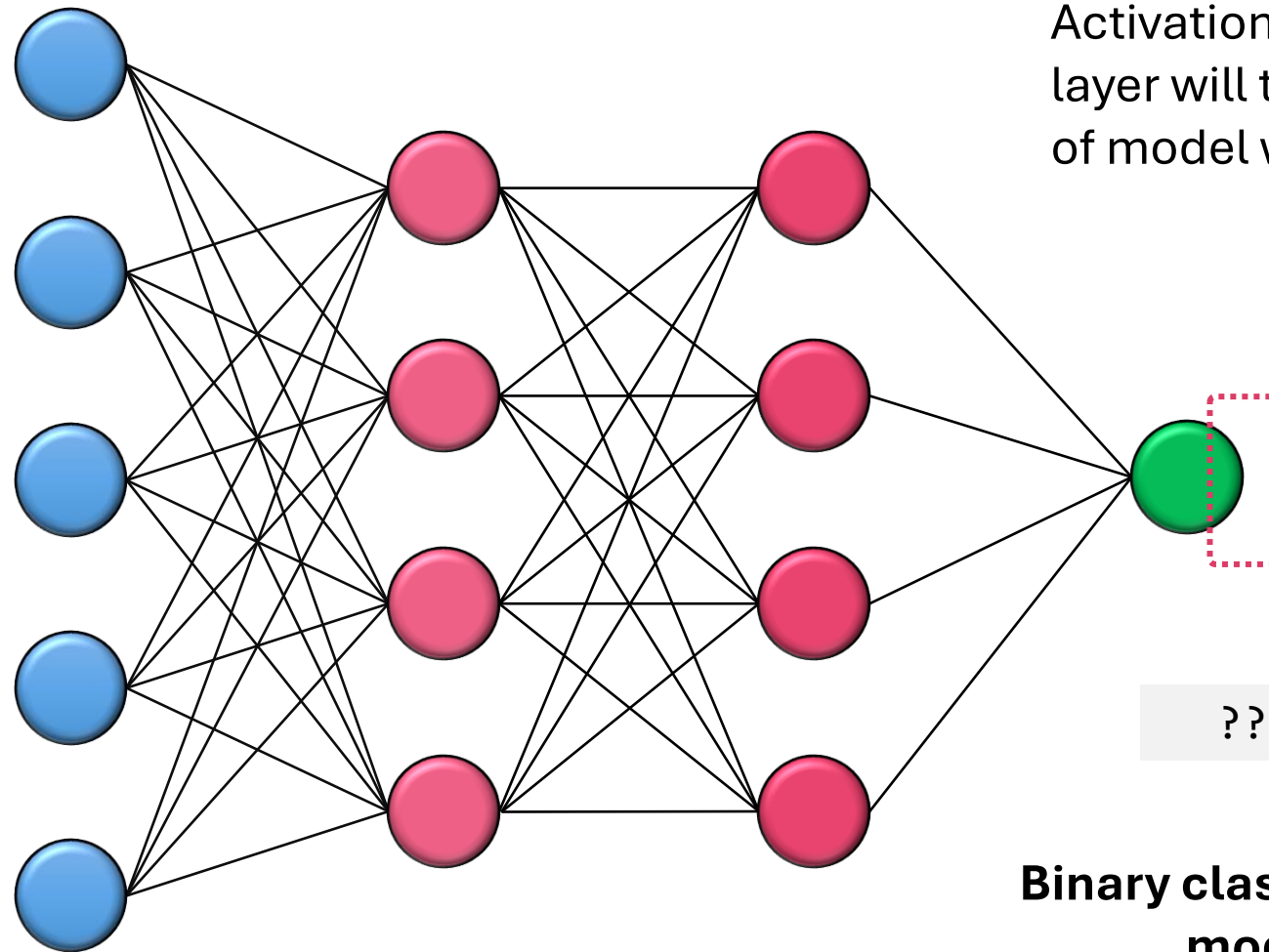
Regression model



Activation function for the output layer will then define which type of model we want

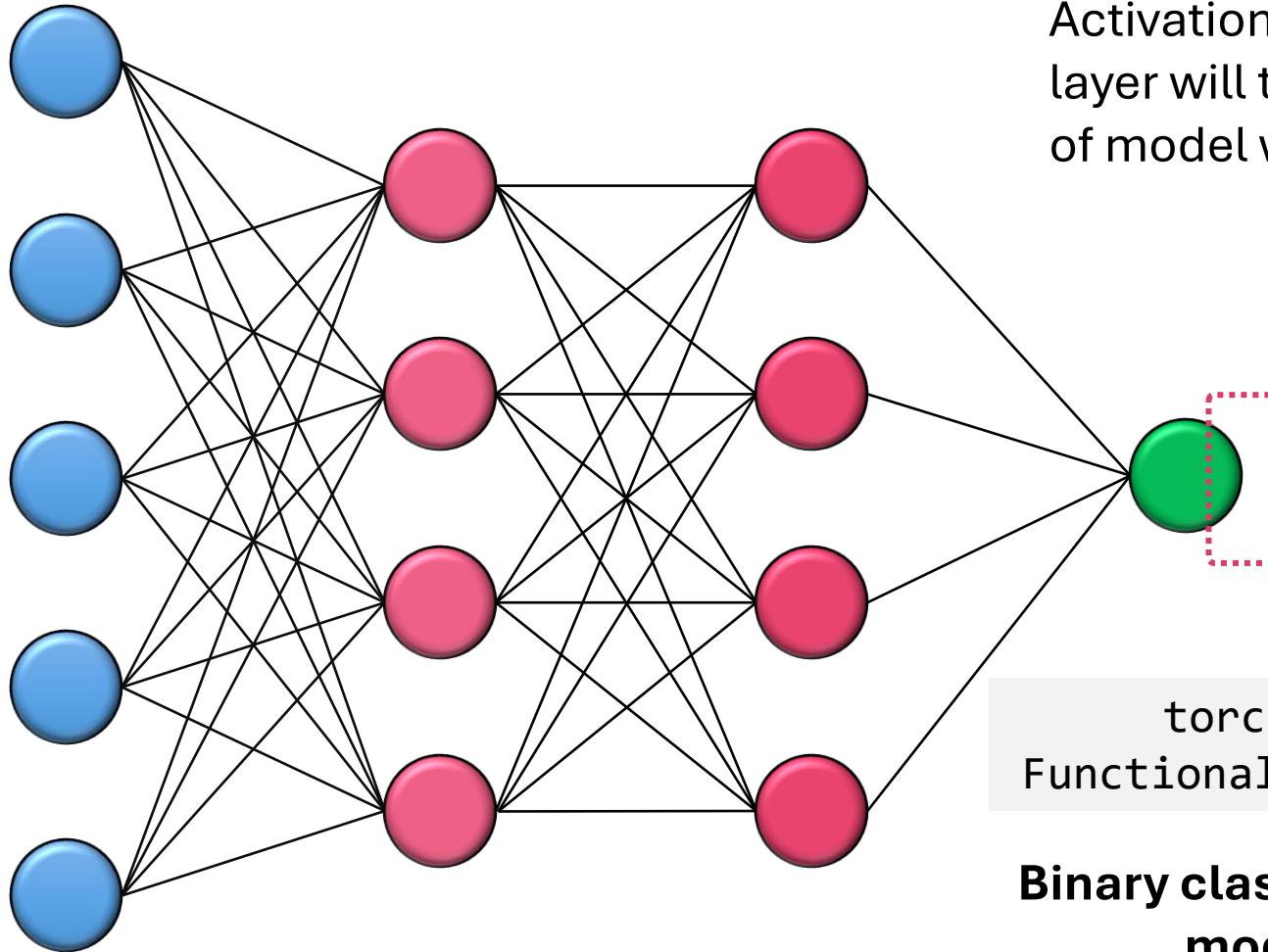
No activation Function

Regression model
 $\Rightarrow \text{float } [-\infty; +\infty]$



Activation function for the output layer will then define which type of model we want

Binary classification model

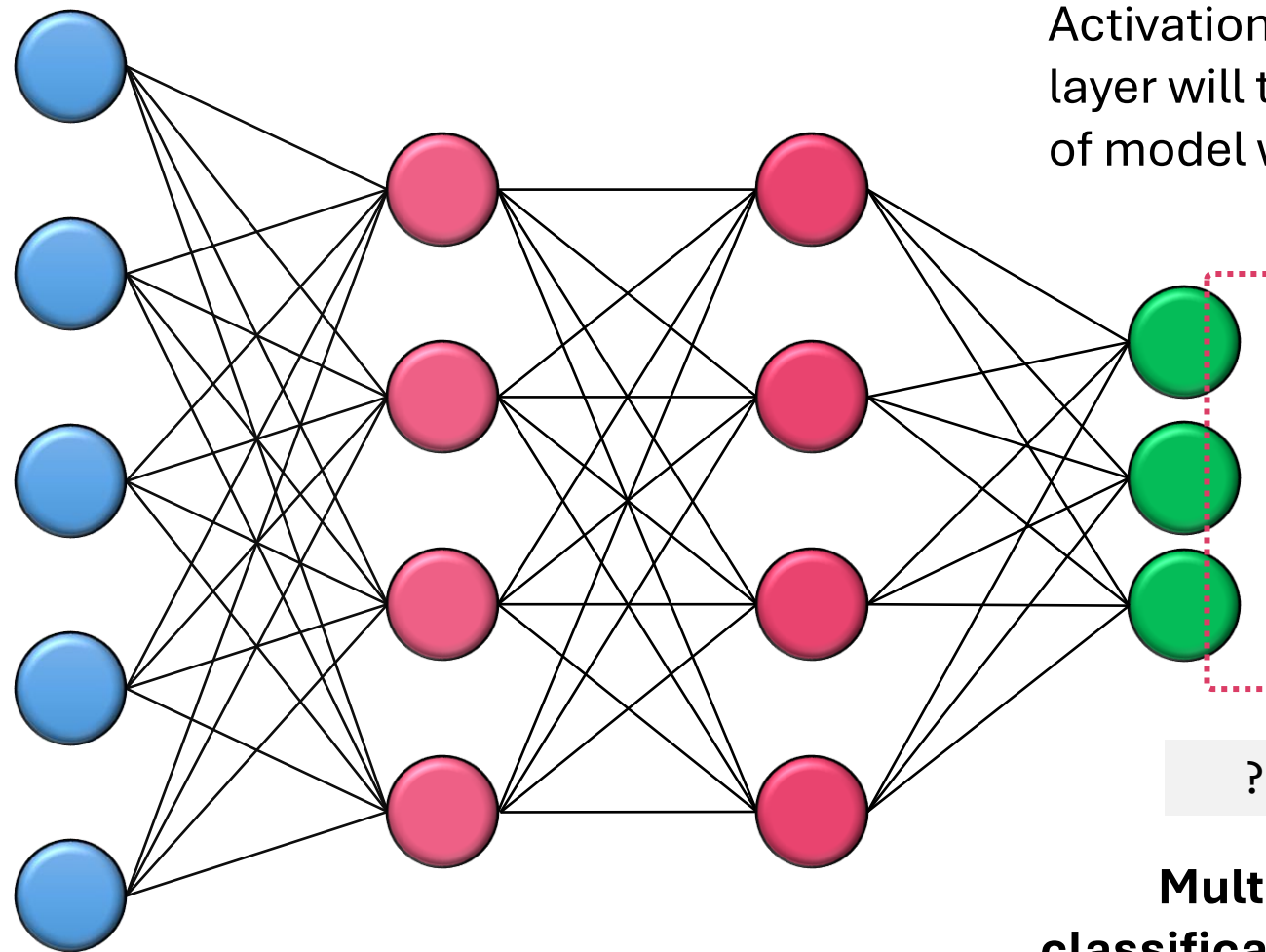


Activation function for the output layer will then define which type of model we want

```
torch.nn.  
Functional.sigmoid()
```

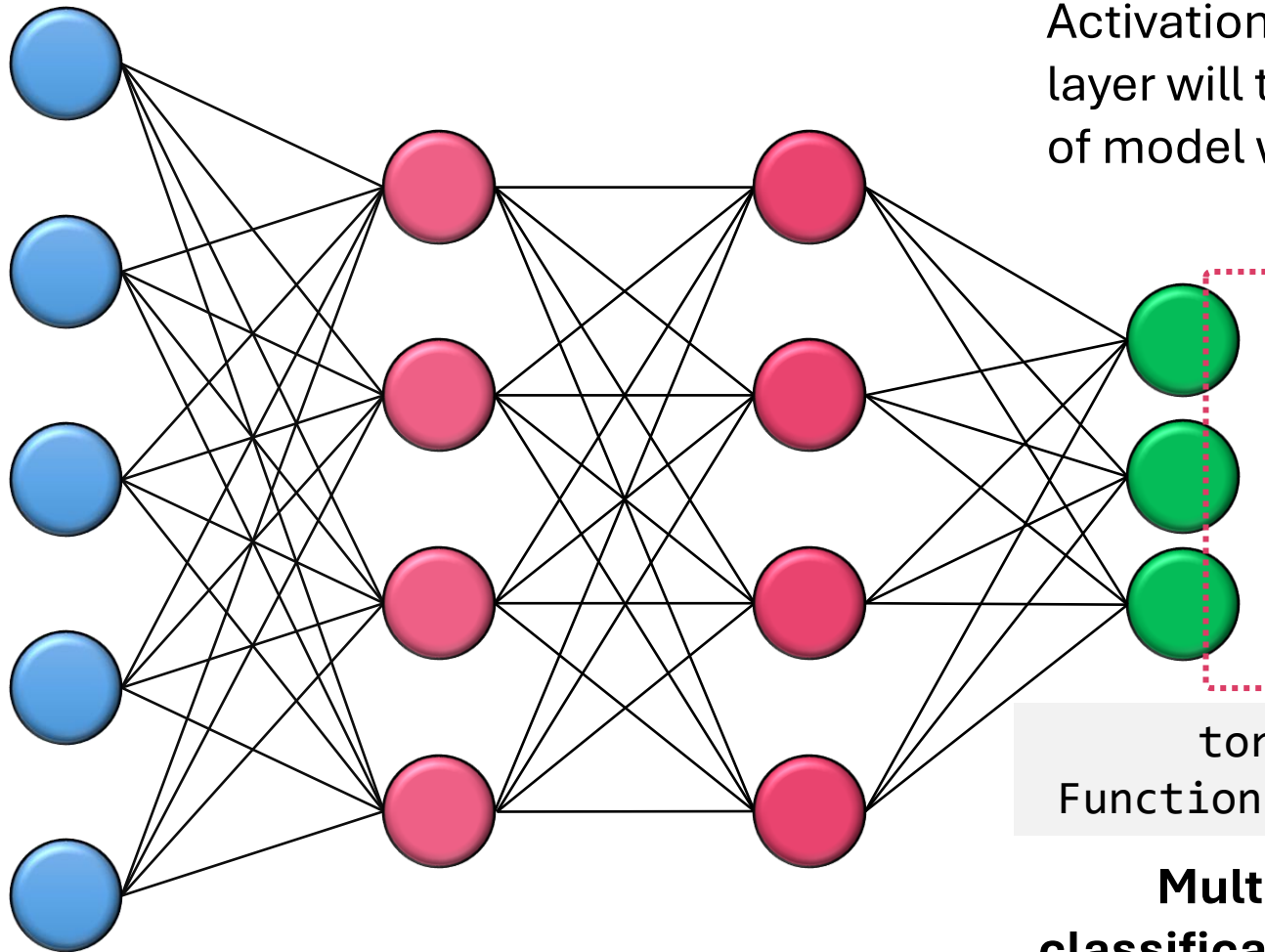
**Binary classification
model**

⇒ probability [0; 1]



Activation function for the output layer will then define which type of model we want

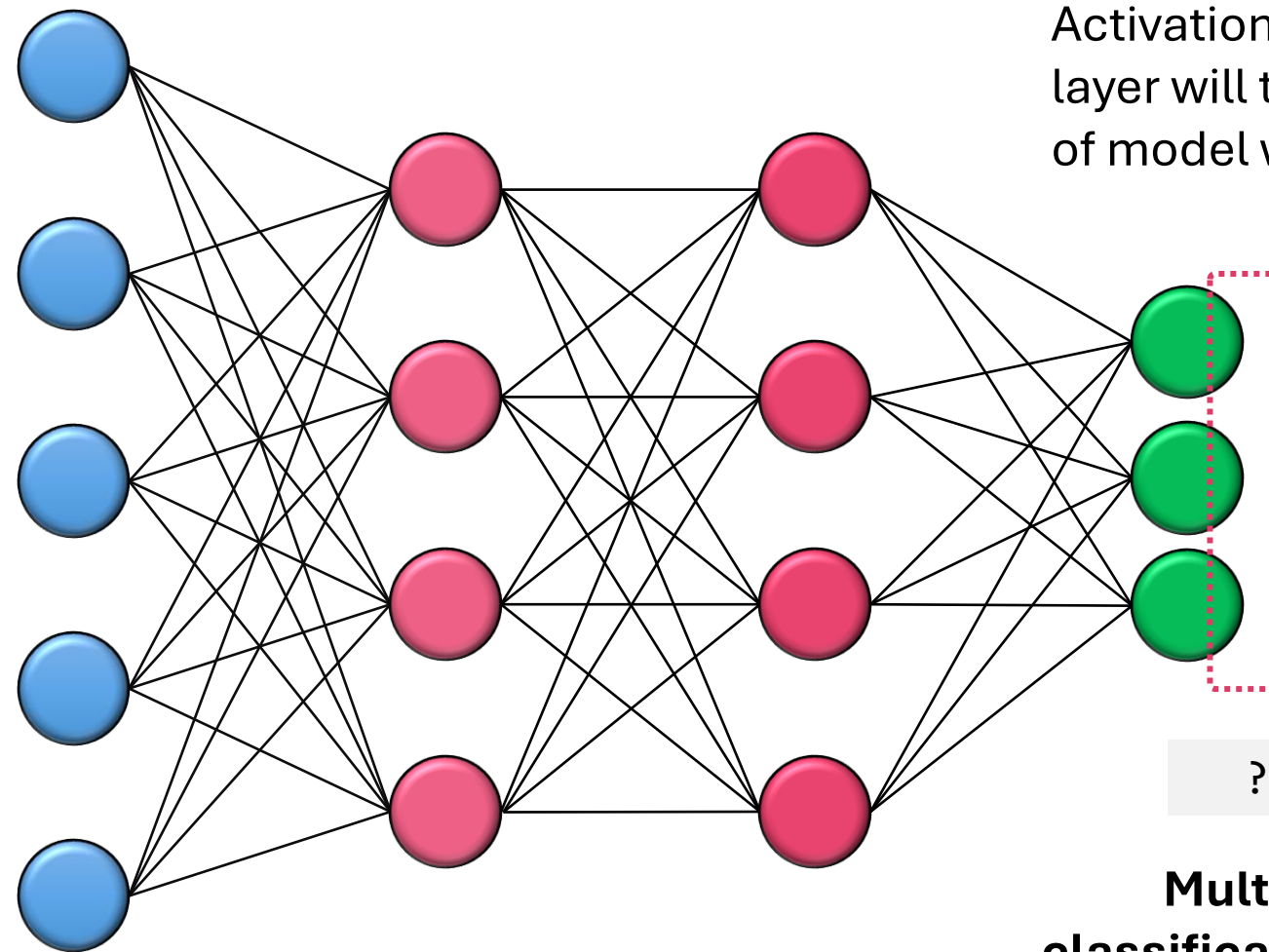
**Multiclass
classification model**



Activation function for the output layer will then define which type of model we want

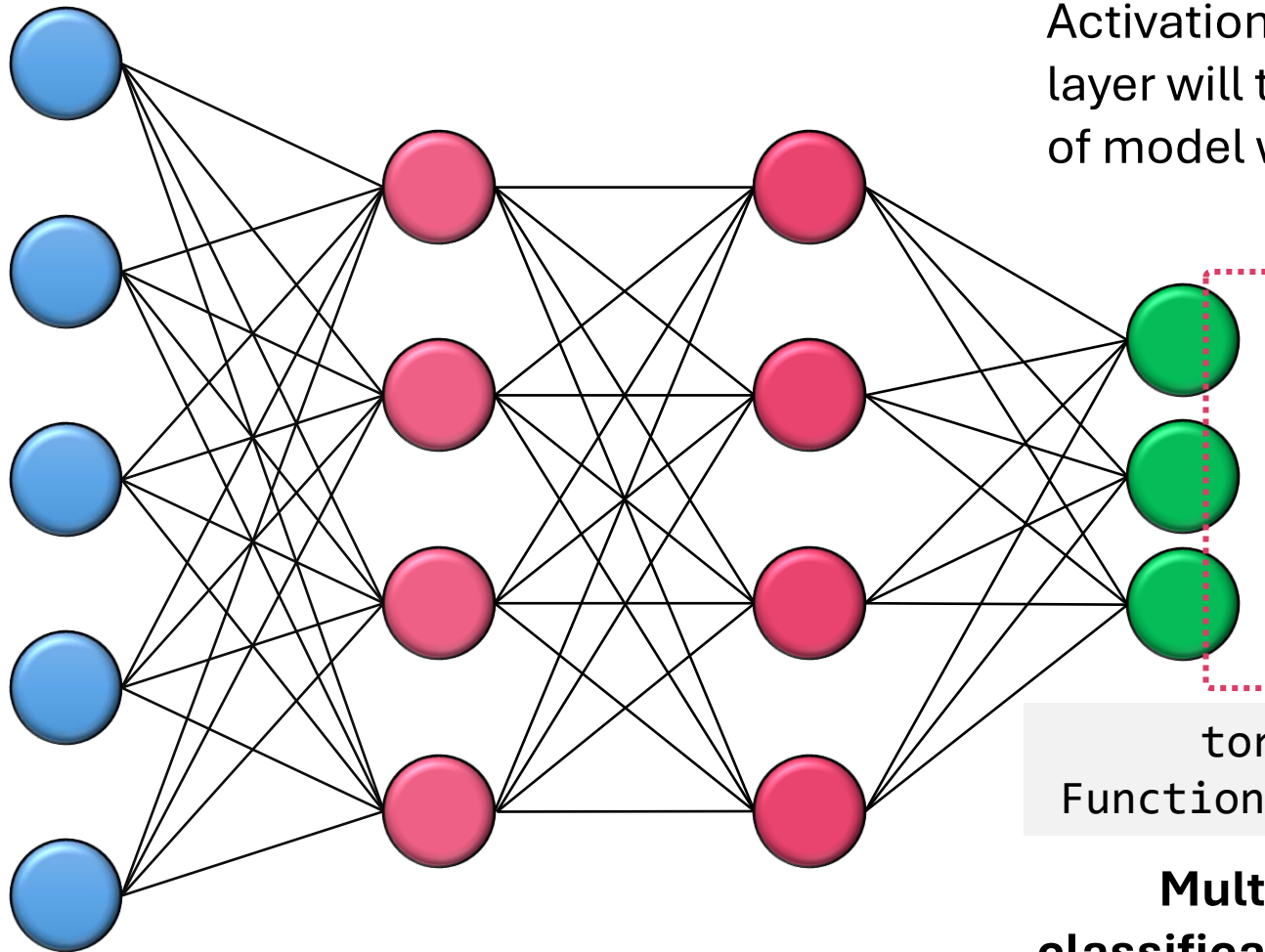
```
torch.nn.  
Functional.softmax()
```

**Multiclass
classification model**
⇒ probabilities [0; 1]
(sum = 1)



Activation function for the output layer will then define which type of model we want

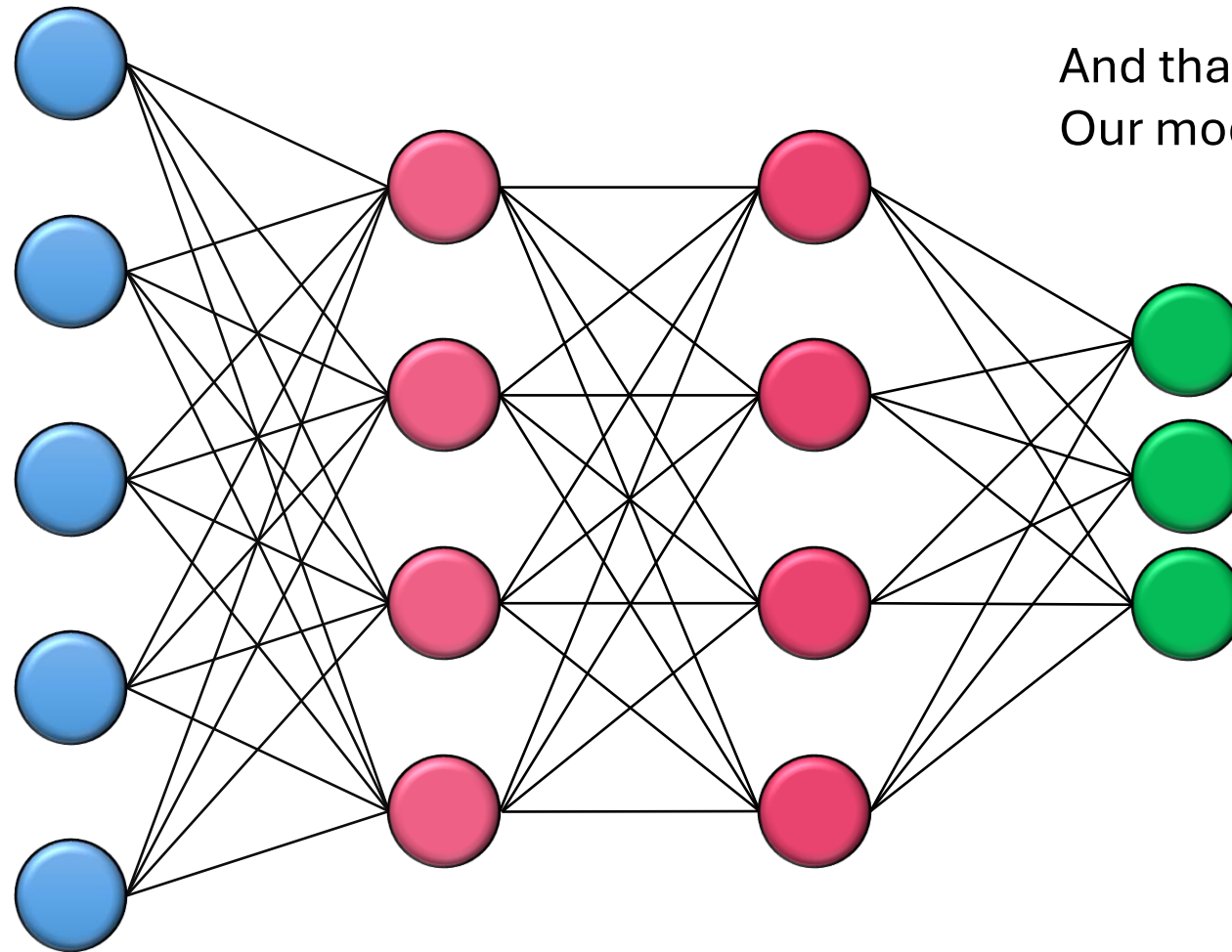
**Multilabel
classification model**



Activation function for the output layer will then define which type of model we want

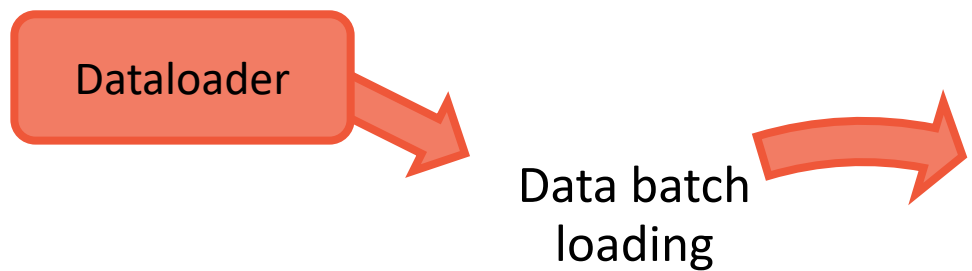
```
torch.nn.  
Functional.sigmoid()
```

**Multilabel
classification model**
⇒ probabilities [0; 1]
(each independent)

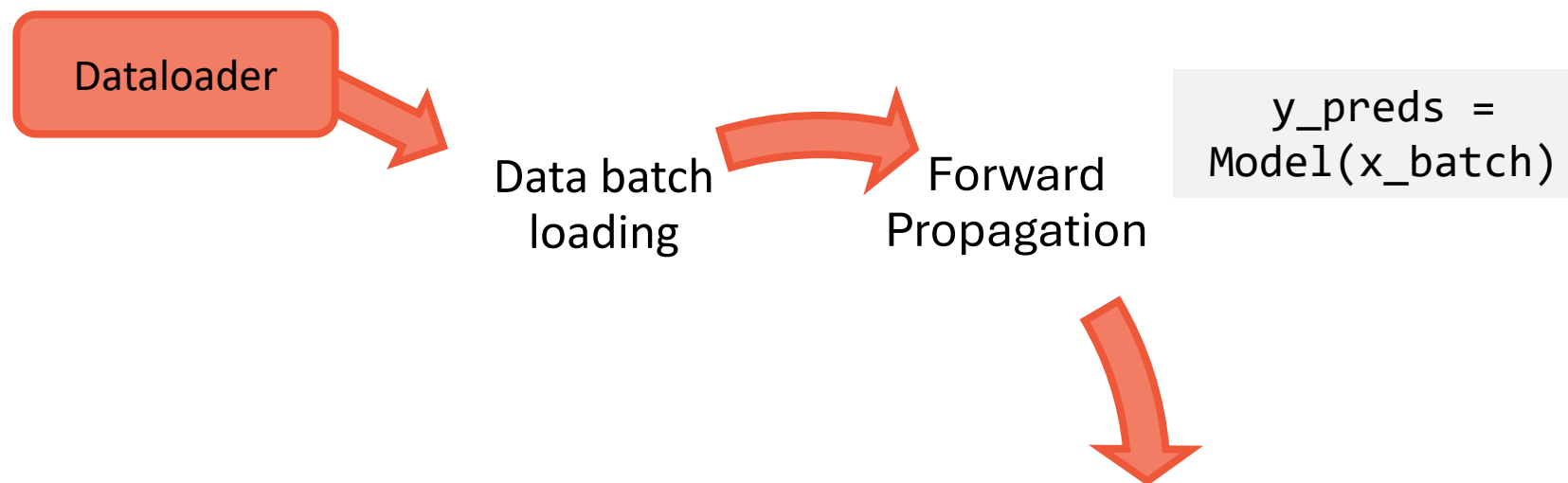


And that's pretty all!
Our model is designed

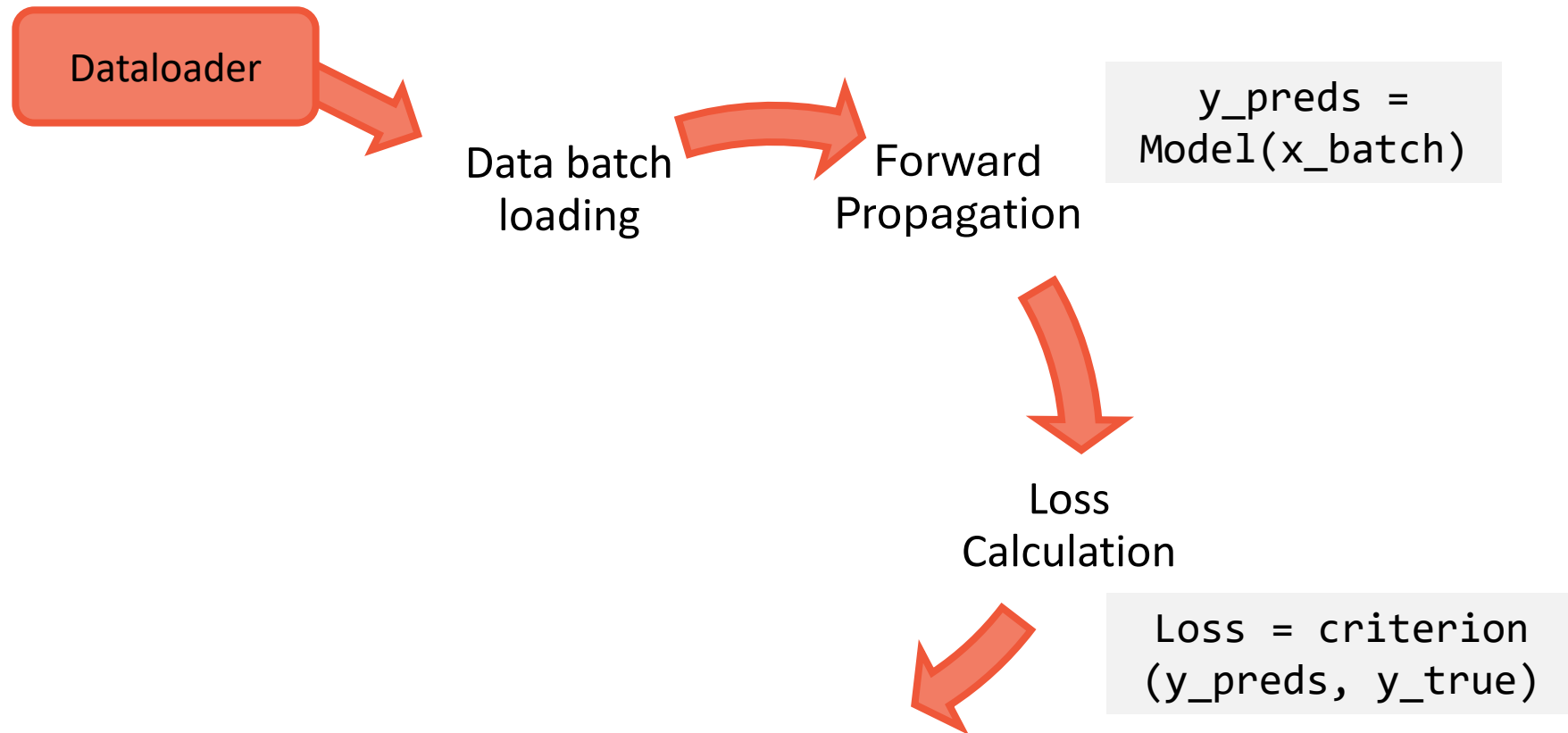
PyTorch training loop



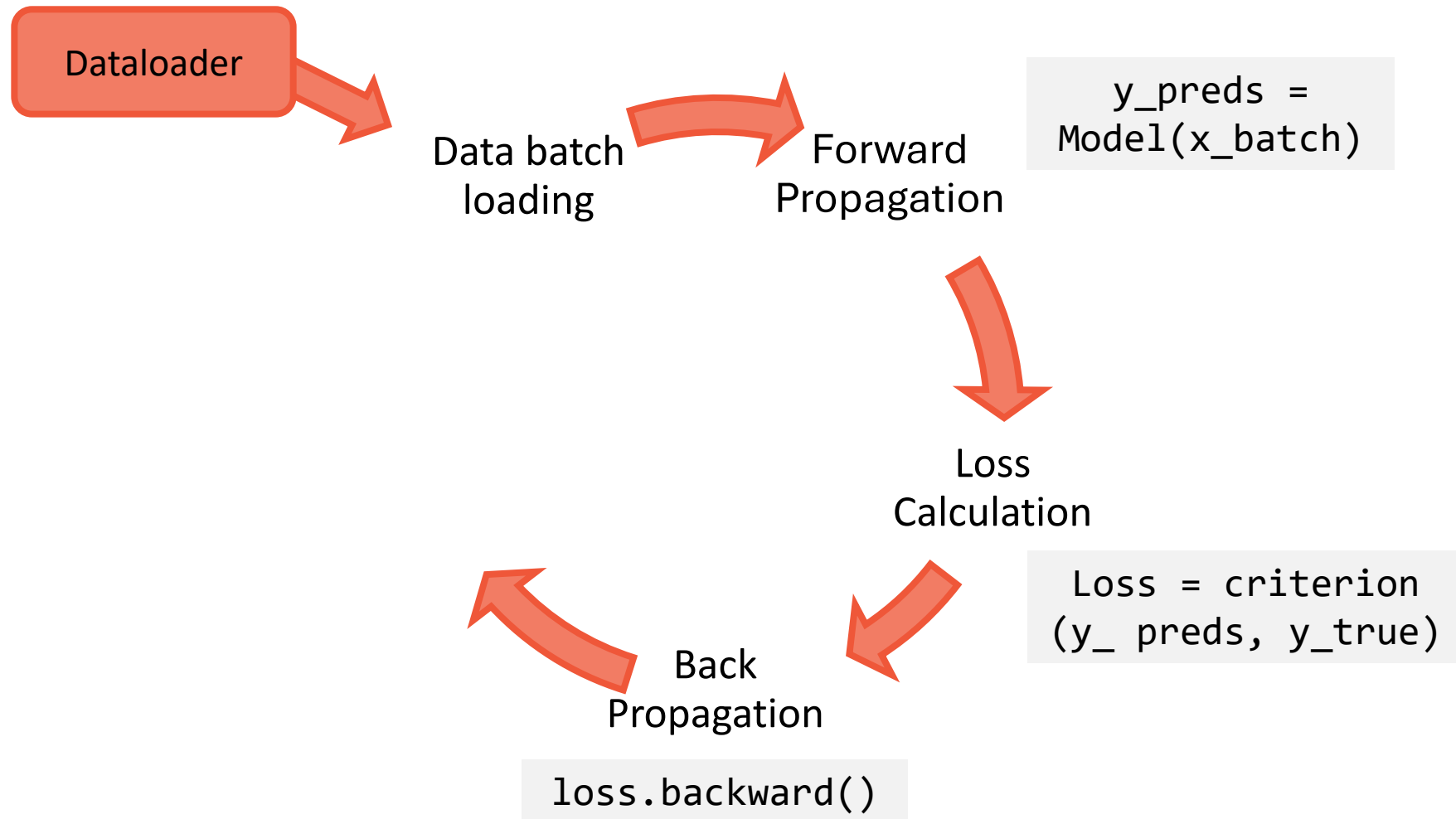
PyTorch training loop



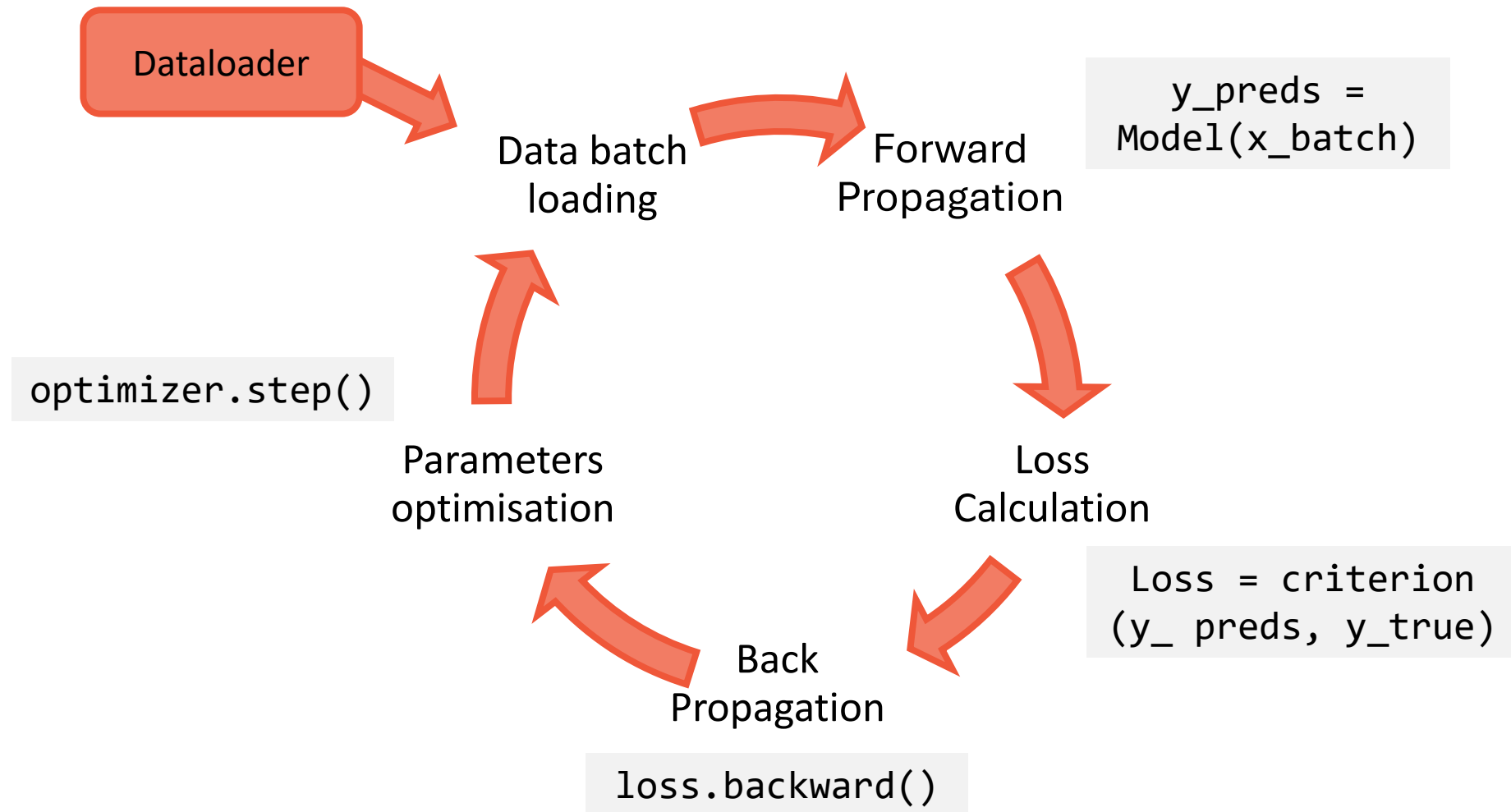
PyTorch training loop



PyTorch training loop



PyTorch training loop



- ❑ Be careful!
- ❑ In PyTorch, `nn.BCEWithLogitsLoss()` internally applies both:
 - Sigmoid, to convert logits to probabilities
 - Loss calculation
- ❑ Thus, the model itself should not contain a sigmoid activation during training !
- ❑ Sigmoid activation should be used only for inference



```
# training
criterion = nn.BCEWithLogitsLoss()
logits = model(X)
loss = criterion(logits, y)
```

```
# Inference
with torch.no_grad():
    logits = model(X)
    probs = torch.sigmoid(logits, dim=1)
    preds = (probs >= 0.5).int()
```


- ❑ Be careful!
- ❑ In PyTorch, `nn.CrossEntropyLoss()` internally applies both:
 - Softmax, to convert logits to probabilities
 - Negative log-likelihood, to compute the loss
- ❑ Thus, the model itself should not contain a softmax layer during training !
- ❑ Softmax layer should be used only for inference



```
# training
criterion = nn.CrossEntropyLoss()
logits = model(X)
loss = criterion(logits, y)
```

```
# Inference
with torch.no_grad():
    logits = model(X)
    probs = torch.softmax(logits, dim=1)
    preds = torch.argmax(probs, dim=1)
```